STACKSAVE

Application Overview, User Guide and Technical Documentation

Android grocery price discovery and comparison application

Document purpose
This guide explains what StackSave is designed to do, how a user navigates its main functions, how its services communicate, and what is required to configure and run the application.

Table of Contents

1. Introduction and purpose 3
2. What StackSave does 3
3. Main application features 4
Search and live price lookup 4
Category discovery 4
Price and offer review 4
Saved deals 4
Shopping list / basket 4
Account and profile 4
Offline support and settings 4
Product images 4
4. How the application works 4
4.1 Search and price retrieval flow 4
4.2 System components 5
4.3 Important data flow and security boundary 5
5. User instructions 5
5.1 Create an account or sign in 5
5.2 Search for a product 5
5.3 Browse by category 6
5.4 Sort and review offers 6
5.5 Save a deal 6
5.6 Build a shopping list 6
5.7 Change settings and sign out 6
6. Installation and configuration 6
6.1 Requirements 6
6.2 Configure Firebase 7
6.3 Configure the backend 7
6.4 Configure the Android backend address 7
6.5 Run the Android app 7
7. Technical architecture 7
7.1 Key Android source areas 7
7.2 Backend request behaviour 8
8. Privacy, security and responsible use 8
9. Troubleshooting 8
The app says Firebase is not configured 8

The backend returns 401 8
The backend returns 403 8
The backend returns 429 8
The backend returns 502 or no results 9
The app cannot connect to localhost 9
Port 8080 is already in use 9
Product images do not appear 9
Gradle cannot download dependencies 9
10. Current implementation notes and limitations 9
11. Use of Artificial Intelligence (AI) in Application Development 10
12. Glossary 10

1. Introduction and purpose

StackSave is an Android application concept that helps shoppers discover grocery products, retrieve retailer offers, compare prices and organise items they intend to buy. Its central aim is to make grocery shopping more informed by bringing product and retailer price information into one mobile experience.

Instead of checking several retailer websites or applications separately, a user can search for a product such as milk, bread or rice and review the retailer offers returned by the connected grocery catalogue service. The user can then assess the displayed prices, save interesting deals and add products to a shopping list.

Core value proposition
Search once, compare available retailer offers, and plan a shopping list with clearer visibility of prices.

2. What StackSave does

The application combines the following activities:

- Product discovery: search for a grocery item using a text query.
- Retailer price comparison: view matching offers from retailers returned by the connected AZ Labs grocery API.
- Category discovery: select a product category to trigger a relevant catalogue search.
- Deal organisation: save or bookmark products of interest.
- Shopping-list planning: add products to a list, change quantities and review estimated subtotals and totals.
- Account access: register or sign in using Firebase Authentication when Firebase is correctly configured.
- User preferences: manage available profile settings, including language, notification preference and offline-sync preference.
- Product imagery: display an image when the catalogue response supplies a usable image URL; otherwise, show a fallback image/placeholder.

StackSave is a price-discovery and planning tool. It does not itself sell groceries, process retailer payments, reserve stock or guarantee that a displayed price is still available at a particular branch.

3. Main application features

Search and live price lookup

The user enters a product or search phrase. The app sends the request through the StackSave backend. The backend queries the AZ Labs grocery endpoints and returns the response to the Android app. The app converts supported response fields into product and retailer offer cards.

Category discovery

Category choices such as Groceries, Fresh Food, Household, Personal Care and Baby & Family are mapped to representative search terms. Selecting a category starts a new API search. These are search terms sent to the existing endpoint, not separate category endpoints supplied by AZ Labs.

Price and offer review

Offer cards show the product name, retailer, returned price and any supported promotion or stock details. Where the API provides a previous/original price, the app can use it to show a comparison. If no previous price is supplied, the displayed current price should not be interpreted as a verified discount.

Saved deals

Users can bookmark or unbookmark an offer to keep it in their saved-deal collection during the app session. The current source should not be assumed to provide cloud-synchronised saved deals unless that behaviour is implemented and tested separately.

Shopping list / basket

Products can be added to the list. Quantity controls adjust the number of units, and the interface calculates estimated item subtotals and a combined estimated total from the available prices.

Account and profile

Firebase Authentication supports email-and-password registration and sign-in when the app is connected to the intended Firebase project. Profile settings are stored through the user repository and Firestore when configured. A clearly labelled demo sign-in is a local demonstration and is not a real Firebase session.

Offline support and settings

The project includes an offline repository/cache and a setting for offline synchronisation. The availability and freshness of cached information depend on the cache implementation and whether data has previously been loaded. Cached prices must not be treated as guaranteed current prices.

Product images

The app attempts to load image URLs supplied by the product API. Image availability depends on the provider response and retailer image-host permissions. A missing image URL cannot be generated by the app itself.

4. How the application works

4.1 Search and price retrieval flow

1. The user opens StackSave and signs in with a Firebase-enabled account.
2. The user types a product name or selects a category.
3. The Android ViewModel passes the search request to DealFeedService.

4. DealFeedService obtains a Firebase Authentication ID token for the signed-in user.
5. The Android app sends the search request and token to the StackSave Node.js backend.
6. The backend verifies the Firebase ID token using Firebase Admin SDK.
7. The backend sends a request to AZ Labs using the private AZ Labs API key stored on the server.
8. The backend tries the grocery comparison endpoint and, when needed, the grocery search endpoint.
9. The returned JSON is sent back to the Android app. DealFeedService maps supported product, retailer, price, image, promotion and stock fields into app data objects.
10. The interface displays the returned offers and the user can review, save or add products to the shopping list.

4.2 System components

Component	Technology	Responsibility
Android application	Kotlin, Jetpack Compose	Screens, navigation, search input, product cards, saved deals, basket and settings.
Application state	StackSaveViewModel, Kotlin StateFlow	Maintains screen state and coordinates actions between UI and repositories.
Authentication	Firebase Authentication	Registers users, signs users in and provides ID tokens.
User profile storage	Cloud Firestore	Stores supported user profile and preference data.
Backend proxy	Node.js, Express	Verifies Firebase tokens and forwards authorised grocery searches to AZ Labs.
Grocery data provider	AZ Labs API	Supplies product and retailer offer data for requested searches.
Product images	Coil image loader	Loads remote image URLs when included in API responses.
Local data	OfflineRepository / local storage	Supports local caching and selected app preferences.

4.3 Important data flow and security boundary

The AZ Labs API key belongs on the backend, not in the Android source code or a publicly distributed APK. The mobile app sends a Firebase ID token to the backend. The backend verifies that token before using its server-side API credential to request catalogue data. This reduces the risk of exposing the provider key.

Important
The live price search flow requires a real Firebase-authenticated session. The local Demo Login option is for interface demonstration only and does not provide the Firebase ID token required by the protected backend.

5. User instructions

5.1 Create an account or sign in

11. Launch StackSave.
12. Choose the registration option if you are a new user.
13. Enter the requested name, email address and password, then submit the form.
14. For an existing account, enter the registered email and password and sign in.
15. If authentication fails, confirm that Email/Password is enabled in the connected Firebase project and that the app's google-services.json belongs to package com.example.stacksave.

5.2 Search for a product

16. Open the Deals or home discovery area.
17. Tap the search field and enter a specific product name, for example 'milk', 'bread' or 'rice'.

18. Select Compare live prices or the relevant search action.
19. Wait for the loading indicator to finish.
20. Review the returned retailer cards. Check the retailer name, price, promotion text and any stock information.
21. Use refresh or change the search term if no matching offers are returned.

5.3 Browse by category

Open the Category selector and choose a category. StackSave maps that choice to representative search words and sends them to the same live catalogue search service. For example, Fresh Food uses terms such as chicken, eggs, vegetables and fruit. Results depend on what the provider returns for those terms.

5.4 Sort and review offers

Use the available sort selector to organise the displayed offers, for example by lowest price or biggest discount where the required data exists. Compare like-for-like products: package size, unit quantity, brand and product variant can affect whether two prices are directly comparable.

5.5 Save a deal

22. Find the product card you want to keep.
23. Tap its bookmark/save control.
24. Open the saved-deals or favourites area to review saved items.
25. Tap the save control again to remove an item from the saved collection.

5.6 Build a shopping list

26. Choose a product from the results and use the add-to-list/basket action available in the interface.
27. Open My List or the basket area.
28. Use the quantity controls to increase or decrease the number of units.
29. Review each item subtotal and the estimated combined total.
30. Remove an item when it is no longer required.

The basket total is an estimate based on the prices currently held by the app. It may not include delivery fees, retailer-specific charges, substitutions, promotions that require eligibility, or price changes at checkout.

5.7 Change settings and sign out

Open the account/profile area to review the available settings. The source includes language, notification and offline-sync preferences. Change a preference and allow the app to save it. Use Sign Out to end the Firebase session on the device.

6. Installation and configuration

6.1 Requirements

- Android Studio with a compatible Android Gradle Plugin and Kotlin/Compose toolchain.
- JDK 17.
- Android emulator or physical Android device; the project sets minSdk to 24.
- Internet access for Gradle dependency resolution, Firebase services and live catalogue requests.
- Node.js and npm for the backend.
- Firebase project with the Android app registered as com.example.stacksave.
- AZ Labs API access and a valid API key.

6.2 Configure Firebase

31. Open Firebase Console and select the Firebase project intended for StackSave.
32. Register an Android app with package name com.example.stacksave if it is not already registered.
33. Download the project's google-services.json file.
34. Place it at StackSave2/app/google-services.json, replacing any configuration for a different Firebase project.
35. Enable Email/Password under Firebase Authentication sign-in providers.
36. Create a Cloud Firestore database and apply restrictive security rules for authenticated users.
37. Sync and rebuild the Android project. Confirm the Firebase project ID shown by the app is the intended project.

6.3 Configure the backend

Open a terminal in the StackSave2/backend folder and create a private .env file with the following settings:

```
PORT=8080
AZ_LABS_API_KEY=YOUR_PRIVATE_AZ_LABS_KEY
FIREBASE_SERVICE_ACCOUNT_JSON=./firebase-service-account.json
```

Place the Firebase Admin service-account JSON file at the path specified by FIREBASE_SERVICE_ACCOUNT_JSON. The service account must belong to the same Firebase project used by the Android app. Keep both .env and the service-account JSON private; do not commit them to source control or include them in a public ZIP.

Install backend dependencies and start the service:

```
npm install
npm start
```

The backend listens on port 8080 by default. Its health endpoint is /health. A healthy configuration should report that the service is running, Firebase Admin is initialised and the AZ Labs key is configured.

6.4 Configure the Android backend address

In the project root local.properties file (the same level as settings.gradle.kts), retain the existing Android SDK setting and add the backend URL. For the Android emulator, use:

```
STACKSAVE_BACKEND_BASE_URL=http://10.0.2.2:8080/
```

10.0.2.2 is the special emulator address for reaching the development computer's localhost. A physical device needs the computer's reachable LAN address and appropriate firewall/network configuration. The app uses cleartext HTTP for this local development setup; production deployments should use HTTPS.

6.5 Run the Android app

38. Open the StackSave2 project folder in Android Studio.
39. Allow Gradle to sync and download required dependencies.
40. Confirm the Firebase configuration and backend URL are correct.
41. Start the Node backend and confirm its /health endpoint.
42. Start an emulator or connect an Android device with USB debugging enabled.
43. Select the app run configuration and press Run.
44. Register/sign in with Firebase, then search for a product to test the full API flow.

7. Technical architecture

7.1 Key Android source areas

File or folder	Purpose

MainActivity.kt	Application entry point and Compose screen/navigation host.
ui/StackSaveViewModel.kt	Coordinates screen state, searches, categories, saved offers, basket, account and settings actions.
ui/screens/	Compose screens for home/deals, account and product-related experiences.
ui/components/ProductImage.kt	Reusable product image display component.
model/StoreDeal.kt	Data model representing a product/retailer offer.
repository/DealFeedService.kt	Gets Firebase ID token, calls backend and maps API JSON into StoreDeal objects.
repository/StackSaveBackendApi.kt	Retrofit interface for the StackSave backend endpoint.
repository/AuthRepository.kt	Firebase authentication operations.
repository/UserRepository.kt	User profile persistence.
repository/OfflineRepository.kt	Local cache and offline-related support.
backend/server.js	Express API proxy, Firebase token verification and AZ Labs requests.

7.2 Backend request behaviour

The Android Retrofit client calls GET /api/deals with an Authorization: Bearer <Firebase-ID-token> header and search and limit query parameters. The backend verifies the token, then requests AZ Labs /api/v1/grocery/compare and falls back to /api/v1/grocery/search when appropriate. The backend limits the requested result count and does not return the private API key to the Android client.

8. Privacy, security and responsible use

- Never place the AZ Labs private API key in Kotlin source, local Android resources or a public APK.
- Do not share the Firebase Admin service-account JSON file or backend .env file.
- Use the same Firebase project for the Android google-services.json and backend service account.
- Configure Firestore security rules so users can access only the records they are authorised to access.
- Use HTTPS for any production backend endpoint. The included HTTP emulator URL is for local development.
- Treat retailer prices and stock as time-sensitive. Confirm the final price and availability with the retailer before purchase.
- Respect AZ Labs and retailer terms governing data use, caching, attribution and product imagery.
- Collect only the personal information needed for account and application functions.

9. Troubleshooting

The app says Firebase is not configured

Check that app/google-services.json exists, matches com.example.stacksave and belongs to the intended Firebase project. Sync Gradle and rebuild.

The backend returns 401

Sign in using Firebase rather than Demo Login. Confirm the Android app and Firebase Admin service account use the same project. Sign out and sign in again to refresh the ID token.

The backend returns 403

Check the AZ_LABS_API_KEY value in backend/.env and confirm the key is valid for the AZ Labs API.

The backend returns 429

The provider may have rate-limited the request or the API quota may have been reached. Wait and retry or check the provider account.

The backend returns 502 or no results

Check internet connectivity, the backend terminal logs, the provider key and the exact AZ Labs endpoints/parameters enabled for the account. Try a more specific product term.

The app cannot connect to localhost

For an Android emulator use http://10.0.2.2:8080/. Confirm the backend is running and Windows Firewall permits the connection.

Port 8080 is already in use

Find the process listening on port 8080 and stop the duplicate process, or configure the backend and Android base URL to use another matching port.

Product images do not appear

Inspect the raw API response for imageUrl, image_url, image or thumbnail. Some products may not include an image URL, and some retailer hosts block external image loading.

Gradle cannot download dependencies

Check internet/DNS access, proxy settings and the configured Gradle distribution. Retry Gradle sync when the distribution host is reachable.

10. Current implementation notes and limitations

This documentation was prepared by inspecting the supplied StackSave_Category_Discovery project archive. The source contains a Firebase-authenticated AZ Labs proxy flow, category-to-search mapping, product offer mapping, saved-deal state, basket quantity state, account/profile operations and offline repository support.

Verification status

The project archive's README states that a Gradle build could not be completed in the previous review environment because services.gradle.org was unreachable. This document therefore describes the inspected source and intended operating flow; it does not certify that every feature has passed a clean build, device test or production test.

The following limitations should be considered before presenting StackSave as a production-ready shopping service:

- Retailer catalogue coverage depends on the AZ Labs subscription, endpoints, search terms and response fields.
- A successful API response does not guarantee that every product has an image, promotion, previous price or stock status.
- The application does not complete purchases or retailer checkout.
- The basket is an estimated planning tool; the displayed total is not a final payable amount.
- Some preferences and cached information may be local or session-based; confirm persistence behaviour with tests before promising cross-device synchronisation.
- The source includes a demo sign-in path. It must not be used to test protected live API calls because it does not create a Firebase session.
- Before release, run Gradle sync/build, test registration and sign-in, verify the backend health endpoint, test live searches, check empty/error states and validate the privacy/security configuration.

11. Use of Artificial Intelligence (AI) in Application Development

Artificial Intelligence (AI), particularly ChatGPT, assisted me throughout the development of the StackSave application. It was used as a development support tool to help generate and improve Kotlin code, implement features, refine the user interface, troubleshoot errors, and resolve technical issues in Android Studio.

AI also assisted with testing and debugging by helping identify possible coding errors, suggesting solutions, and explaining how to test application functionality, including Firebase authentication, API integration, product searches, basket management, and checkout processes. It provided guidance on running the application, configuring dependencies, and resolving build-related issues.

Although AI contributed to the development process, I remained responsible for reviewing the suggested code, applying changes, testing the application, and ensuring that the implemented features met the project's requirements. This helped improve my programming knowledge, problem-solving abilities, and understanding of the software development process.

12. Glossary

Term	Meaning
API	Application Programming Interface; a defined way for software systems to exchange data.
AZ Labs	The external grocery catalogue/data provider used by the StackSave backend.
Backend proxy	A server that receives app requests and securely forwards them to another service.
Firebase Authentication	A service used to register users and authenticate sign-in sessions.
Firebase ID token	A signed token that the backend verifies to confirm the user's Firebase identity.
Firestore	Firebase's cloud NoSQL database used by the project for supported user profile data.
Jetpack Compose	Android's declarative UI toolkit used to build the StackSave interface.
Kotlin StateFlow	A state holder used to expose changing application data to the user interface.
Retailer offer	A product and price entry returned by a retailer catalogue source.
Unit price	A price normalised to a standard quantity, useful for comparing different package sizes.

End of documentation